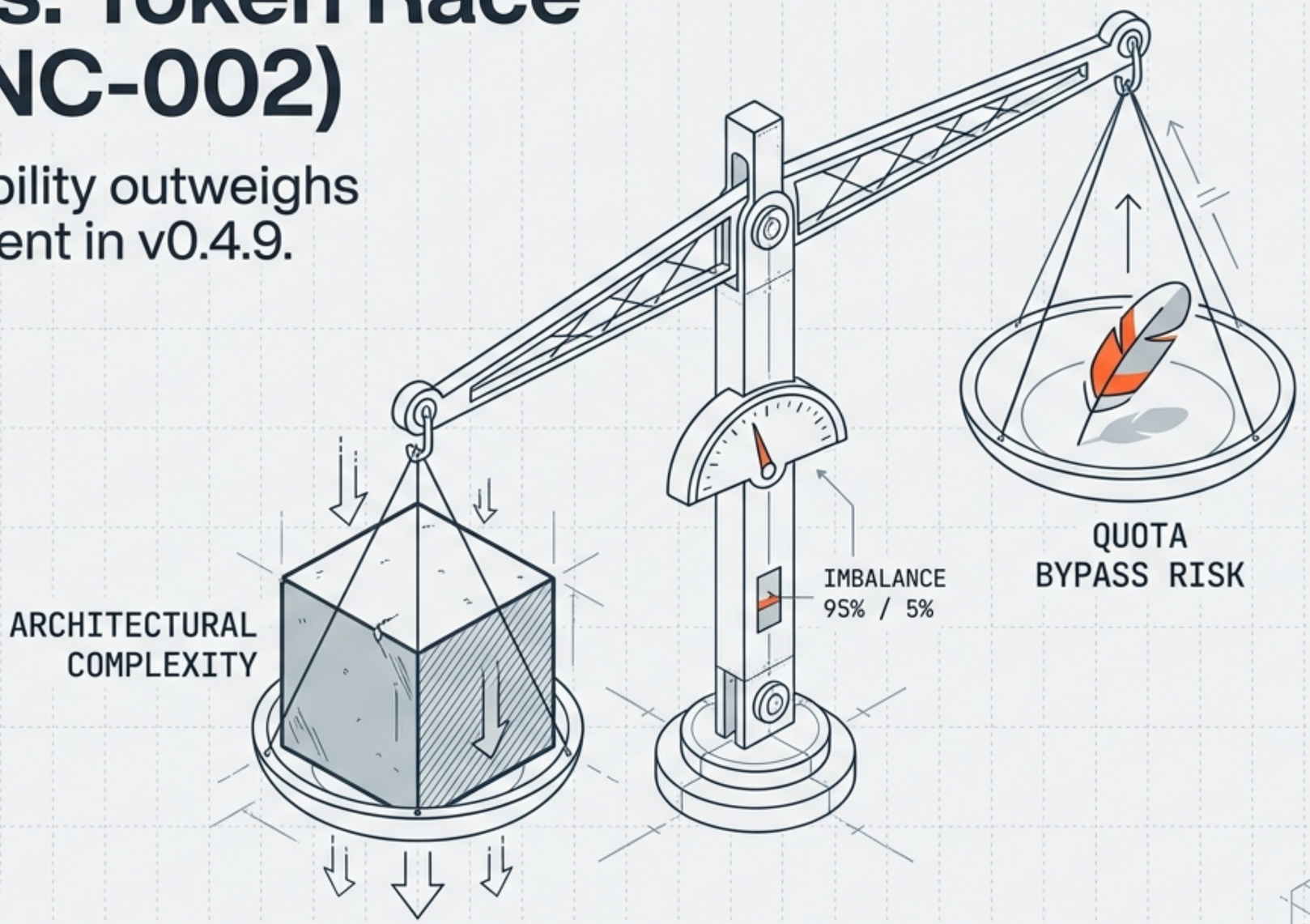
Risk Analysis: Token Race Condition (INC-002)

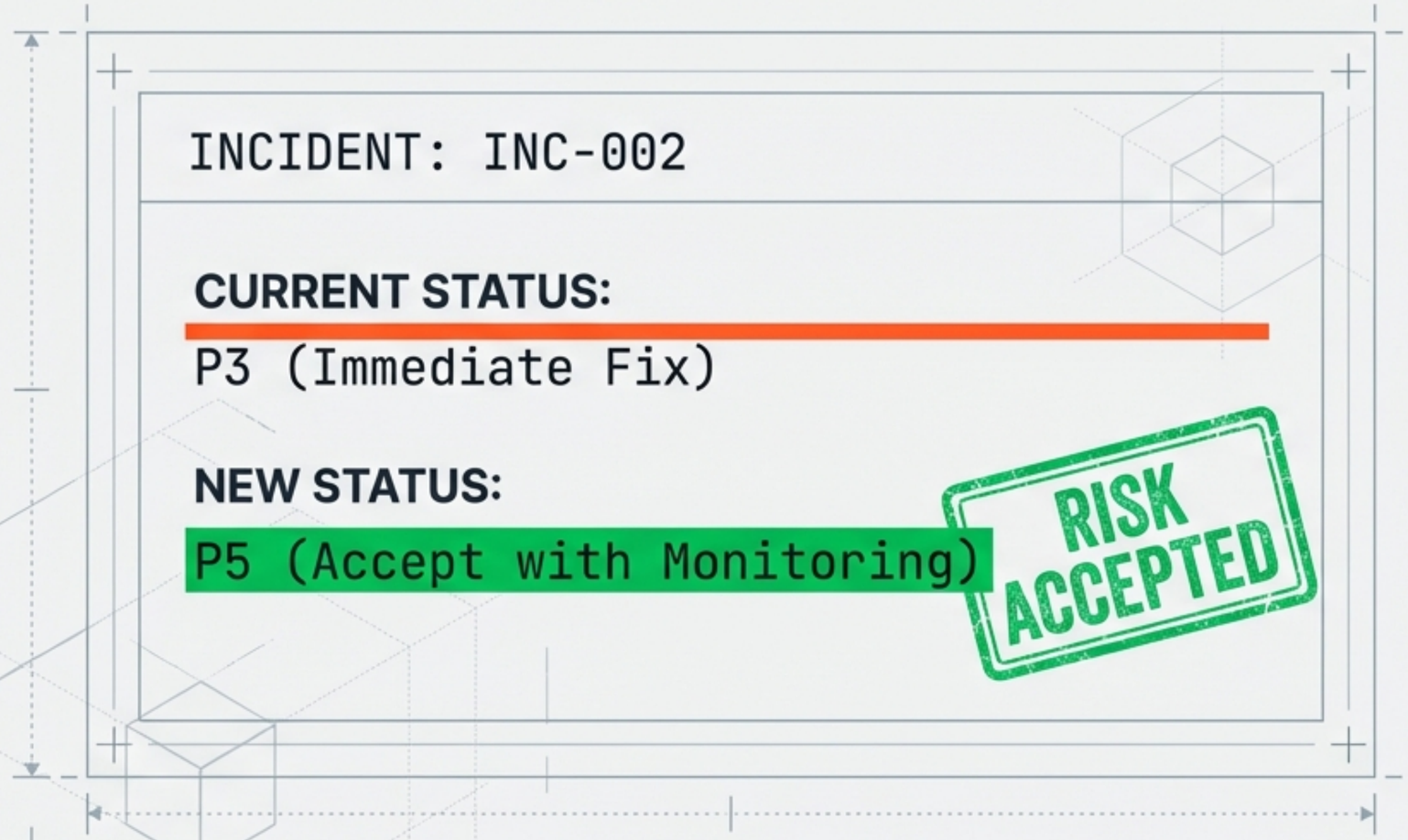
Why architectural stability outweighs strict quota enforcement in v0.4.9.



DATE: 17 February 2026
VERSION: v0.4.9
AUTHORS: GRC (Lead), AppSec, Architect

Recommendation: Reclassify to P5 (Informational) and Accept Risk

The current solution (fixing the bug) is operationally riskier than the problem itself.

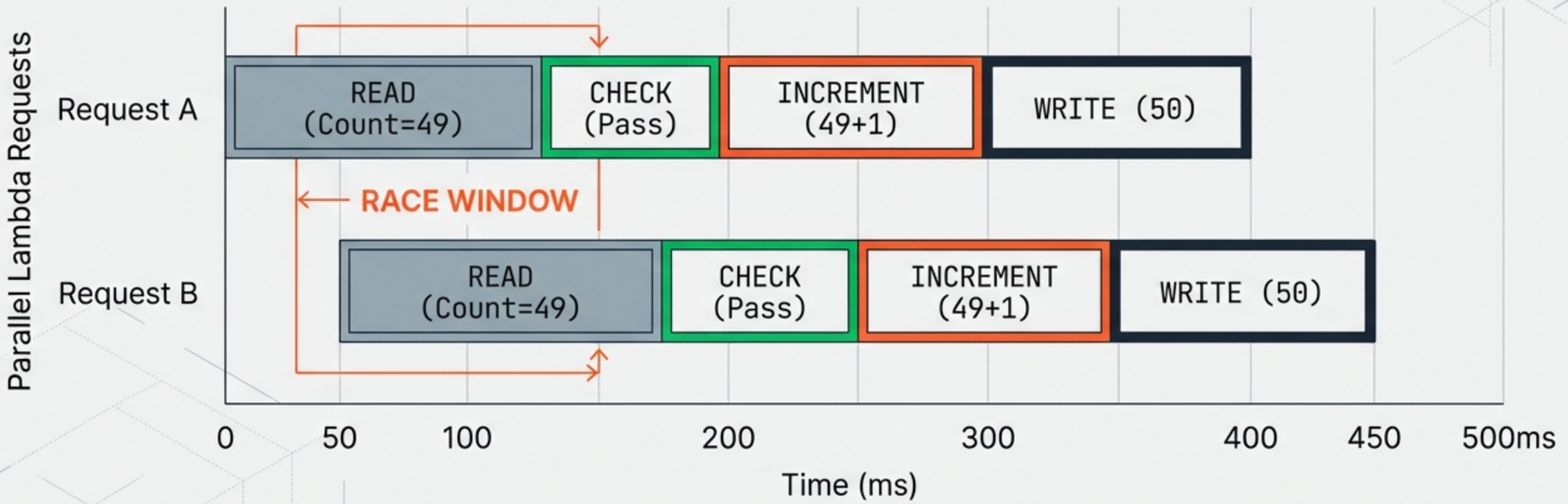


THE RATIONALE:

1. **Complexity**: A proper fix introduces new failure modes and dependencies.
2. **Maturity**: The product stage does not warrant an architectural overhaul for this edge case.
3. **Safety**: User data remains encrypted and secure regardless of quota bypass.

Anatomy of the Race Condition

The 40-400ms Window



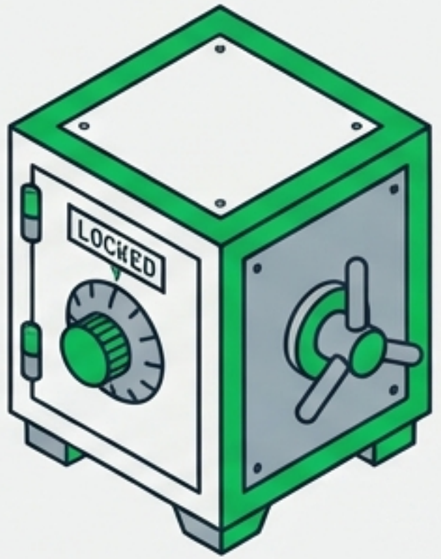
Expected: 51 | Actual Result: 50

Impact is Quota Bypass, Not Data Breach

COMPROMISED

- Accurate Usage Counts
- Audit Trail Precision
- Usage Limits

SECURE



- User Data (Encrypted Bytes)
- Decryption Keys (Never stored on server)
- Admin Access

“Ciphertext without a key is worthless.”

Threat Scenarios: The Malicious User & The Stolen Token

The Malicious User

Action: Scripts 20 concurrent requests.

Result: Quota exceeded (~65 uploads).

Impact: 15 extra encrypted files stored.

Assessment: Nuisance. No Data Breach.

The Stolen Token

The Real Problem: Token theft allows 50 valid uploads.

The Bug's Contribution: Adds ~10 extra uploads.

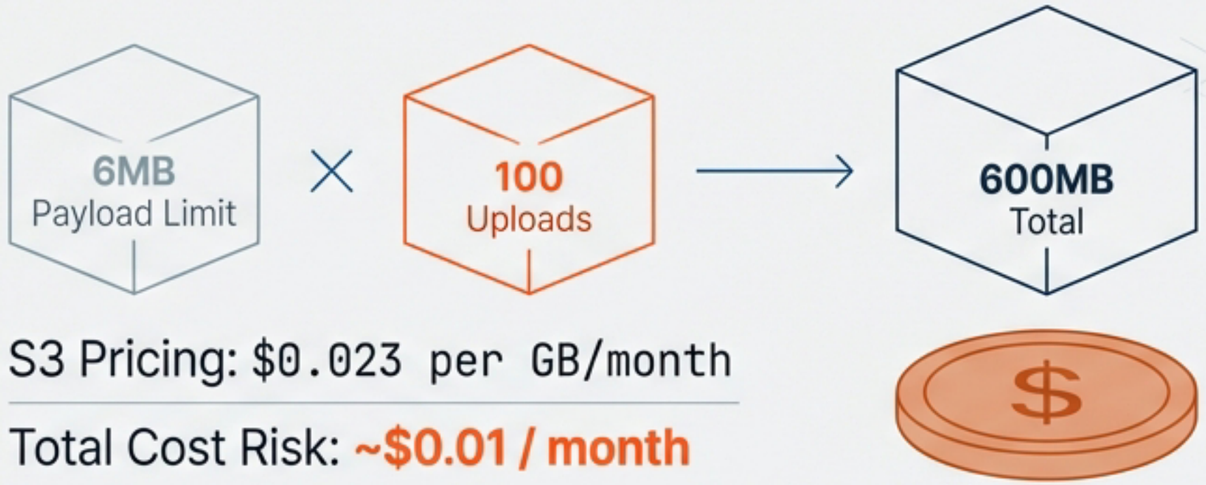
Assessment: The catastrophe is the stolen token, not the race condition.

The race condition provides negligible marginal gain to an attacker who already holds a valid token.

Why Cost Exhaustion and Malware aren't Drivers for a Fix

Cost Analysis (The Math)

Scenario: 100 Extra Uploads



S3 Pricing: `$0.023 per GB/month`

Total Cost Risk: **~$0.01 / month**

Malware Distribution

Does the bug enable malware distribution?

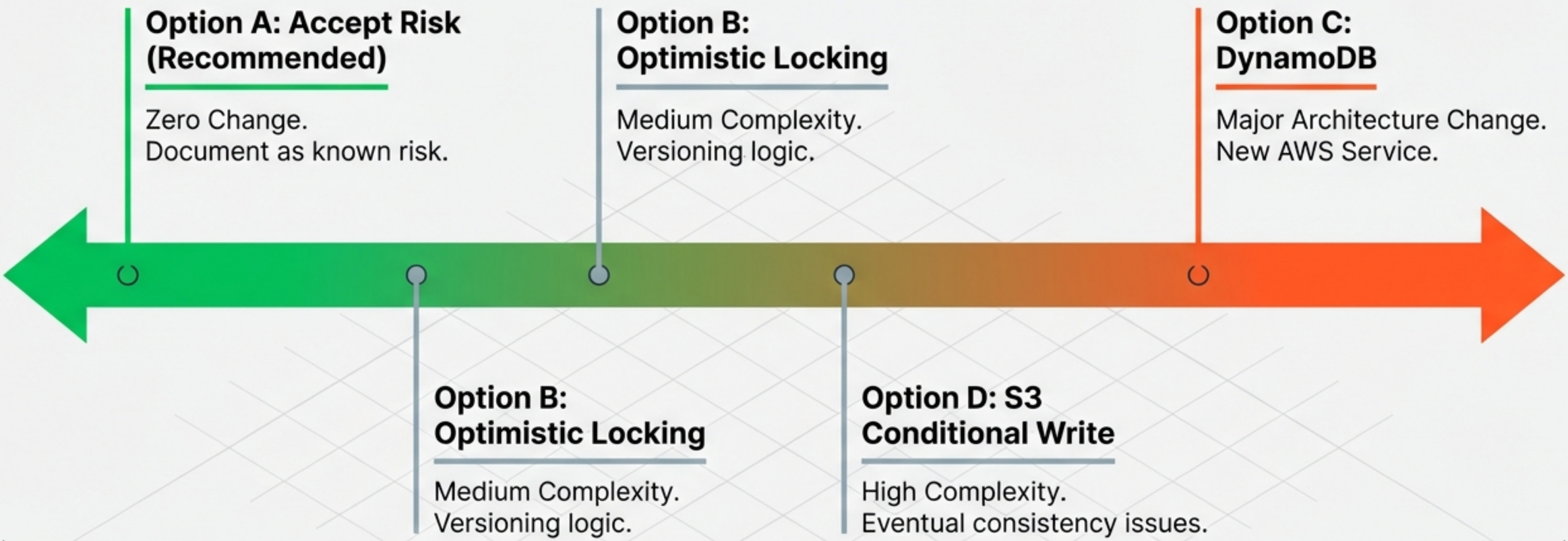
[Valid Token] → [Allows 50 Uploads (Malware)] → [Baseline Risk]

[Valid Token + Bug] → [Allows 60 Uploads (Malware)] → [**No Change** in Threat Profile]

Mitigation belongs in **Abuse Detection**, not counting.

The Fix Options Spectrum

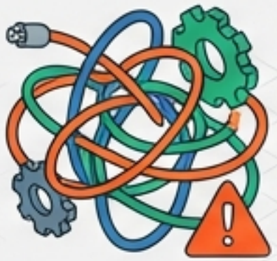
From Zero Change to Major Surgery



The Hidden Costs of Complexity

Why we rejected Options B, C, & D

Option B (Locking)



Technical Debt

Risk: Retry Storms

- Potential for infinite loops under load.
- Unknown cache support for Conditional Updates.

Option C (DynamoDB)

AWS Service Sprawl

Risk: Architecture Sprawl

- Adds entire new AWS Service for one counter.
- Requires new IAM Roles & Migration.
- Breaks ‘No Mocks’ testing principle.

Option D (S3 Write)

Eventual Consistency

Risk: False Security

- Eventual consistency means fix is not guaranteed.
- High operational burden.

Assessing Risk of Vulnerability vs. Risk of Fix

Metric	Option A (Accept)	Option B (Locking)	Option C (DynamoDB)
New Failure Modes	0 (Zero)	2-3 (Retry Storms)	5+ (Throttling, IAM)
New Dependencies	0	0	1 (New Service)
Implementation Effort	None	Medium	High
Data Breach Risk	None	None	None

Option A is the only choice with **ZERO** new failure modes.

Aligned Across Disciplines

AppSec

Fix risk exceeds vulnerability risk. Current impact is limited to Quota Bypass (P4–P6).

Architecture

Don’t bolt on DynamoDB for a single counter. Fix it upstream in the production storage layer. **Simplicity is a feature.**

GRC

Reclassify to P5. Monitoring is a sufficient control for the current risk level.

Acceptance is Not Ignorance

Monitoring Strategy

Log Terminal >_

```
[INFO] Service: TokenManager | Event: QuotaExceeded | Count: 52 | Limit: 50 | Action: LOG_ONLY
```

Action: Log line added when `usage_count > usage_limit`

Trigger: Alert on anomalous patterns (e.g., >1000% of limit)

Result: Operational awareness without architectural complexity.

Future State Re-evaluation

When do we fix this?

Monetisation

When quotas are directly tied to revenue (Customer Tiers).

Storage Migration

Move to Production Architecture (S3 + DynamoDB). Atomic operations become native.

Simplicity is a security feature.

We retain it for now.